Contrôle de redondance cyclique

But du TD/TP : écrire une application qui permet de calculer des sommes de contrôles.
Cf. : https://fr.wikipedia.org/wiki/Contr%C3%B4le_de_redondance_cyclique

Note : il vous faudra faire générer les classes d'exceptions

1. Reprenez l'interface de polynôme suivante :

```
public interface Polynome {
    /**
     * Le degré du polynome : 1 pour x+1, 2 pour x**2+x+1
     * @return le degré
     */
    int getDegre();

    /**
     * Recupère le i-ème coefficient, pour 2x + 3, 0->3, 1->2
     * @param i le rang
     * @return le coefficient
     */
    int getCoeff(int i);

    /**
     * Définit le i-ème coefficient, pour 2x + 3, 0->3, 1->2
     * @param i le rang
     * @param coeff le coeff
     */
    void setCoeff(int i, int coeff);

    /**
     * Affiche le polynome en forme 3x**2 + 2x + 1
     * @return la chaine
     */
    String affichage();

    /**
     * Affiche le polynome sous une forme binaire pour x**2 + 1 : 101
     * @return la chaine ou null si il y a des coeffs > 1 ou < 0
     * @throws NonBinaireException si le polynome n'est pas binaire
     */
    String affichageBinaire() throws NonBinaireException;

    /**
     * Le polynome est-il nul
     * @return true s'il est nul
     */
    boolean estNul();

    /**
     * Initialise le polynome depuis une chaine binaire 101 pour x**2+1
     * @param chaineBinaire la chaine binaire
     * @throws NonBinaireException si la chaine n'est pas binaire
     */
    void initialize(String chaineBinaire) throws NonBinaireException;
}
```

2. En TDD, codez un test pour cette interface et une implantation.

3. Reprenez l'interface de calcul polynomial suivante :

```
public interface PolyCalc {
    /**
     * Définit le polynôme A
     * @param  polyA le polynôme A
     */
    void setPolyA(Polynome polyA);

    /**
     * Renvoie le polynôme A
     * @return le polynôme A
     */
    Polynome getPolyA();

    /**
     * Définit le polynôme B
     * @param polyB le polynome B
     */
    void setPolyB(Polynome polyB);

    /**
     * Renvoie le polynôme B
     * @return le polynôme B
     */
    Polynome getPolyB();

    /**
     * Effectue une addition des polynomes A + B
     */
    void ajouter();

    /**
     * Ajouter binaire (utilise xor = somme sans retenue)
     */
    void ajouterBinaire();

    /**
     * Effectue une soustraction des polynomes A - B
     */
    void soustraire();

    /**
     * Effectue une multiplication des polynomes A * B
     */
    void multiplier();

    /**
     * Multiplier binaire (utilise une boucle de xor = somme sans retenue)
     */
    void multiplierBinaire();

    /**
     * Effectue une division polynomiale normale : A / B
     */
    void diviser();

    /**
     * Effectue une division polynomiale binaire sans retenue (pour les CRC) : A / B
     * @throws NonBinaireException si l'un des 2 polynômes a un coeff non binaire
     */
    void diviserBinaire() throws NonBinaireException;
```

```
    /**
     * Permet de récupérer le résultat
     * @return le résultat (ou le quotient pour la division)
     */
    Polynome getResultat();

    /**
     * Permet de récupérer le reste
     * @return le reste
     */
    Polynome getReste();
}
```

4. En TDD, codez un test pour cette interface de calcul et une implantation.

5. Reprenez l'interface de calculs de CRC suivante :

```
public interface CRC {
    /**
     * Définit le polynome générateur
     * @param generateur le polynome générateur
     */
    void setPolynomeGenerateur(Polynome generateur);

    /**
     * Modifie le polynome pour que la division par le polynome générateur donne un
reste nul
     * @param donnees le polynome représentant les données
     */
    void metAJour(Polynome donnees) throws NonBinaireException;

    /**
     * Vérifie que la division du polynome reçu par le polynome générateur est nulle
     * @param recu le polynome reçu
     * @return true si le reste est nul
     */
    boolean verifie(Polynome recu) throws NonBinaireException;
}
```

6. En TDD, codez un test pour cette interface de calcul et une implantation.

   Voilà des exemples de calculs et leur résultat :

   Soit le polynôme générateur : 101 (x**2 + 1)

   Soit le message suivant : 0100000101000010

   Le message avec la somme de contrôle est : 010000010100001011

   Si on vérifie ce message on doit obtenir true alors que pour le message avec un bit modifié :
   0100000110100001011 on doit renvoyer false lors de la vérification.

   Autres calculs :
   Générateur : 1010101, Message : 0100100001000101, Message avec CRC : 010010000100010101010110
   Générateur : 10001, Message : 0100100001000101, Message avec CRC : 010010000100010111101

7. Si vous avez fait des tests unitaires, codez des tests d'intégration et vice/versa.

8. Codez un Main qui permet une exécution de ce type :

```
Veuillez taper le polynome générateur :
101
Veuillez taper le message binaire à envoyer ou FIN :
0100000101000010
Voilà le message à envoyer 10000010100001011
Veuillez taper le message binaire à envoyer ou FIN :
FIN
Veuillez taper le message binaire reçu ou FIN :
10000010100001011
Le message est correct 10000010100001011
Veuillez taper le message binaire reçu ou FIN :
01000001101000010 11
Le message est erroné 01000001101000010 11
Veuillez taper le message binaire reçu ou FIN :
FIN
```